```
[ ]: #functions
```

```
[ ]: A function is a block of reusable code taht perform a specific task.
it helps in breaking a large program into smaller manageable parts.
 function denoted by fun_name
why functions?
.increase code reuablity
.improves readbility and modularity(breaking down a complex system into smaller, independent, and interchangeable parts called modules)
.easier debugging and testing
```

```
[ ]: types of functions
1.built in function
  provided by python( print(),input(),len(),sum()...etc)
2.user defined functions
 created by the programmer/developer using def(define)

def fun_name(p1,p2,p3...pn):
    statements
    return value

.function without input and without return
.function without input and with return
.function with input and without return
.function with input and with return

3.lambda functions
 anonymous ,single line functions are created with lambda
```

```
[ ]: function without input and without return

def fun_name():
    statements

call the function
fun_name()
```

```
[1]: #addition of 2 numbers
def add():
    a=int(input("Enter first number"))
    b=int(input("Enter second number"))
    s=a+b
    print(f"sum of {a} and {b} is {s}")
```

```
[2]: add()
```

```
Enter first number 8
Enter second number 1
sum of 8 and 1 is 9
```

```
[ ]: local variables
which declared inside a funcion but cannot acess outside of the function unless function is returning

global variable
you declare outside of a function and use anywhere in that file
```

```
[ ]: function without input and with return

def fun_name():
    statements
    return values

call the function
v1,v2,...vn = fun_name()
```

```
[3]: def add():
    a=int(input("Enter first number"))
    b=int(input("Enter second number"))
    s=a+b
    return a,b,s
```

```
[4]: add()
```

```
Enter first number 3
Enter second number 2
```

```
[4]: (3, 2, 5)
```

```
[6]: v1,v2,r = add()
```

```
Enter first number 3
Enter second number 2
```

```
[7]:  print(f"sum of {v1} and {v2} is {r}")
```

```
sum of 3 and 2 is 5
```

[8]:
```
r
```

[8]:
```
5
```

[ ]:
```
function with input and without return

def fun_name(v1,v2..vn):
    statements


call the function
direct calling
fun_name(v1,v2....vn)

based on user input
v1 = input()
v2 = input()
...
v1n = input()
fun_name(v1,v2...vn)
```

[10]:
```
def add3(n1,n2):
    s=n1+n2
    print(f"sum of {n1} and {n2} is {s}")
```

[12]:
```
add3(2,3)
```

```
sum of 2 and 3 is 5
```

[19]:
```
a=int(input("Enter first number"))
b=int(input("Enter second number"))
add3(a,b)
```

```
Enter first number 4
Enter second number 3
sum of 4 and 3 is 7
```

[ ]:
```
4.function with input and with return

def fun_name(v1,v2...vn):
    statements
    return value

call the function
direct calling
v1,v2...vn = fun_name(v1,v2....vn)
```

[31]:
```
def add4(v1,v2):
    s=a+b
    return s
```

[32]:
```
a=int(input("Enter first number"))
b=int(input("Enter second number"))
r = add4(a,b)
print(f"sum of {a} and {b} is {r}")
```

```
Enter first number 3
Enter second number 2
sum of 3 and 2 is 5
```

[ ]:

[ ]:
```
# create a function print prime numbers bet given range using all methods
```

[10]:
```
def prime():
    stv = int(input("Enter start value:"))
    spv = int(input("Enter stop value:"))
    for i in range(stv,spv+1,1):
        if(i==1):
            continue
        for j in range(2,i,1):
            if i%j == 0:
                break
        else:
            print(i,end=" ")
prime()
```

```
Enter start value: 2
Enter stop value: 9
2 3 5 7
```

[13]:
```
def prime2():
    p1=[]
    stv = int(input("Enter start value:"))
    spv = int(input("Enter stop value:"))
    for i in range(stv,spv+1,1):
        if(i==1):
            continue
        for j in range(2,i,1):
            if i%j == 0:
                break
        else:
            p1.append(i)
    return p1
prime2()
```

```
Enter start value: 2
Enter stop value: 50
```

[13]:
```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

[14]:
```
for i in prime2():
    print(i)
```

```
Enter start value: 2
Enter stop value: 50
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
```